

NOMBRE Y APELLIDOS:

DNI:

FIRMA:

Ejercicio 1

4,75 puntos

Para aprobar el examen es necesario tener 2,65 puntos en este ejercicio.

Implementar las siguientes clases, teniendo en cuenta que:

- Todos los atributos deben tener los modificadores de acceso de tal modo que se favorezca la encapsulación. (-1,0 puntos por cada atributo que no cumpla esta condición)
- Todos los atributos deben tener los métodos de escritura (*setter*) y lectura (*getter*). (-0,50 puntos por cada *setter* o *getter* no implementado)

Todos los métodos que se implementarán en los apartados a) – h) deben ser probados en la clase principal, llamada `PrincipalExamen`. En caso contrario, **se reducirá a la mitad la calificación del apartado que no haya sido probado en la clase principal.**

Clase **Grado**:

Atributos: identificador, nombre, estudiantes matriculados, número máximo de estudiantes, materias del grado y universidad.

- El tipo de atributo de estudiantes matriculados **debe ser de tipo `HashMap`**. (-4,75 puntos si se usa otro)
- Crear un constructor con los siguientes argumentos: el identificador, el nombre y las materias del grado. (-0,50 puntos si no se crea con los argumentos correctos)
- El `HashMap` de estudiantes matriculados se debe recorrer **usando *iterator***. (-0,50 puntos si se recorre de otra forma)

Métodos:

- Matricular a un estudiante** en el grado y en un conjunto de materias dado, teniendo en cuenta el número máximo de estudiantes del grado. (0,50 puntos)
- Dar de baja a un estudiante** en el grado dada la instancia que lo presenta. (0,50 puntos)
- Dar de alta una materia** en el grado dada la instancia que la representa. (0,50 puntos)
- Devolver el **conjunto de estudiantes** del grado matriculados en un curso dado. (0,75 puntos)
- Devolver el **presupuesto** de un grado en función del número total de estudiantes matriculados y del número de estudiantes que han aprobado todas las materias en las que están matriculados. (0,75 puntos)

Clase **Materia**:

Atributos: identificador, nombre, grado, curso y estudiantes matriculados.

- Crear un constructor cuyos argumentos son el nombre, el identificador y el grado. (-0,50 puntos si no se crea con los argumentos correctos)

Clase **Estudiante**:

Atributos: identificador, nombre, tipo, grado, materias en las que está matriculado y calificaciones de las materias.

- Se puede usar cualquier clase para almacenar las materias en las que está matriculado.
- El atributo de calificaciones de las materias **debe ser un `HashMap`**. (-4,75 puntos si es otro tipo)

Métodos:

- Devolver la **nota media de todas las materias** en las que está matriculado el estudiante. (0,50 puntos)
- Devolver el **número de materias de un curso dado** en las que está matriculado el estudiante. (0,50 puntos)
- Indicar **si existe algún estudiante del grado que tenga una nota superior** a la nota del estudiante en una materia dada. (0,75 puntos)

Ejercicio 2

0,75 puntos

Crear un interface llamado `IGrado` que contenga los métodos que están indicados entre los apartados 1.a) – 1.e) y hacer que la clase `Grado` implemente dicho interface (**0,25 puntos**). Además:

- Se deberán modificar las clases `Estudiante` y `Materia` para eliminar la dependencia directa con la clase `Grado`. (**0,50 puntos**)

Ejercicio 3

1,50 puntos

Introducir una excepción asociada al método de matricular a un estudiante, que se lanzará si el número de estudiantes es igual o mayor que el número máximo de estudiantes del grado. Además, la excepción cumplirá las siguientes condiciones:

- Se sacará por pantalla un mensaje sobre la ocurrencia de la excepción. (**0,25 puntos**)
- Si el estudiante que se intenta introducir es de tipo "Erasmus", se admitirá su matrícula siempre que el número de estudiantes sea menor de número máximo de estudiantes + 5. (**0,50 puntos**)

Se considera que una excepción está correctamente introducida si:

- Está bien definida, declarada y se lanza correctamente. (**0,50 puntos**)
- Se prueba correctamente en la clase `PrincipalExamen`. (**0,25 puntos**)

Ejercicio 4

2,75 puntos

Crear una jerarquía de clases en la que aparezcan diferentes tipos de grados y donde se considera que **no** será necesario definir nuevos tipos de grados. Esta jerarquía deberá tener las siguientes características:

- En la clase `PrincipalExamen` se deberán crear dos instancias correspondientes a dos clases de dos niveles diferentes de la jerarquía. (**-0,50 puntos** si no se crean estas instancias)

a) Además de la clase raíz `Grado` la jerarquía deberá contener **dos** niveles de clases: el primer nivel deberá contener **dos** clases, y el segundo nivel deberá contener **dos** clases por cada clase del primer nivel. Además: (**1,25 puntos**)

- La jerarquía de clases debe tener, al menos, una clase abstracta.
- Justificar las decisiones tomadas desde el punto de vista del tipo de clases que se crearán en la jerarquía. Es decir, ¿por qué una clase es abstracta, por qué es final, etc.? Esto se describirá como comentario en la cabecera de la clase `Grado`. (**-0,75 puntos** en caso de que no se realice esta justificación).
- Los nombres de las clases deberán ser consistentes con nombres reales de categorías de grados (no se admiten clases como `grado_1` o `grado_grande`). En el caso de que no sea así, se considerará que la jerarquía **está mal construida**.

b) Cada clase deberá implementar un constructor cuyos argumentos son: `identificador`, `materias` del grado y `nombre`. (**-0,50 puntos** si no se implementa el constructor para cada clase)

- En la implementación del constructor de cada clase se usa la reutilización de código. (**0,25 puntos**)

c) Teniendo en cuenta que el **presupuesto** de un grado depende de su tipo (no es lo mismo un grado que tiene laboratorios o prácticas externas que otro en el que no se realizan prácticas), para cada clase de la jerarquía hay que **sobreescibir el método que calcula el presupuesto** en la clase `Grado`.

- Se considera que los tipos de grados deben tener **todos** unos presupuestos diferentes. (**0,50 puntos**)
- En la sobreescritura del método se hará uso de la reutilización de código entre clases de la jerarquía. (**0,75 puntos** si no se hace con reutilización)